# TALENT HUB

## Database Design and Entity Relationship Model

*Relational Database Specification*

# 1. List of Database Tables

| Sl. No. | Table Name | Purpose |
|---|---|---|
| 1 | students | Stores candidates with USN, branch, CGPA, active backlogs and offer status. |
| 2 | companies | Stores the recruiting organisations. |
| 3 | job_listings | Stores roles posted by companies, with eligibility rules (minimum CGPA, maximum backlogs) and drive date. |
| 4 | job_allowed_branches | Stores the branches eligible for each job (created during 1NF). A job with no rows is open to all branches. |
| 5 | applications | Stores each student's application to a job, with status and timestamps. |
| 6 | interview_slots | Stores interview rounds scheduled for an application. |
| 7 | offers | Stores offers made against an application, with package, offer-letter reference and acceptance. |

**Total number of tables:** 7

# 2. Database Table Details

The following sections define the fields, data types, keys, constraints, and purpose of each table in the Talent Hub relational database. Keys, uniqueness rules, defaults and CHECK rules come from the TalentHub schema; data types and NOT NULL on non-key columns are proposed for implementation.

## 2.1 Students Table

**Purpose:** Stores candidates and the academic data used for eligibility checks.

| Field | Data Type | Key / Constraint | Description |
|---|---|---|---|
| id | INT | PRIMARY KEY, AUTO_INCREMENT, NOT NULL | Unique identifier of the student. |
| usn | VARCHAR(20) | UNIQUE, NOT NULL | University seat number. |
| full_name | VARCHAR(100) | NOT NULL | Full name of the student. |
| email | VARCHAR(150) | UNIQUE, NOT NULL | Email address of the student. |
| branch | VARCHAR(50) | NOT NULL | Academic branch of the student. |
| cgpa | DECIMAL(4,2) | NOT NULL, CHECK (cgpa BETWEEN 0 AND 10) | Current cumulative grade point average. |
| active_backlogs | INT | NOT NULL, DEFAULT 0, CHECK (>= 0) | Number of active backlogs. |
| has_accepted_offer | BOOLEAN | NOT NULL, DEFAULT FALSE | Whether the student has accepted an offer (see Lab 4, Part G). |

## 2.2 Companies Table

**Purpose:** Stores recruiting organisations.

| Field | Data Type | Key / Constraint | Description |
|---|---|---|---|
| id | INT | PRIMARY KEY, AUTO_INCREMENT, NOT NULL | Unique identifier of the company. |
| company_name | VARCHAR(150) | UNIQUE, NOT NULL | Name of the company. |

## 2.3 Job Listings Table

**Purpose:** Stores roles posted by companies and their eligibility rules.

| Field | Data Type | Key / Constraint | Description |
|---|---|---|---|
| id | INT | PRIMARY KEY, AUTO_INCREMENT, NOT NULL | Unique identifier of the job listing. |
| company_id | INT | FOREIGN KEY, NOT NULL | References companies(id); the company posting the job. |

| Field | Data Type | Key / Constraint | Description |
|---|---|---|---|
| role | VARCHAR(100) | NOT NULL | Job role or title. |
| season | VARCHAR(20) | NOT NULL | Placement season of the drive. |
| min_cgpa | DECIMAL(4,2) | NOT NULL | Minimum CGPA required (inclusive). |
| max_backlogs | INT | NOT NULL, DEFAULT 0, CHECK (>= 0) | Maximum active backlogs allowed (inclusive). |
| drive_date | DATE | NOT NULL | Date of the recruitment drive. |
| is_open | BOOLEAN | NOT NULL, DEFAULT TRUE | Whether the job is open for applications. |

## 2.4 Job Allowed Branches Table

**Purpose:** Stores the branches eligible for each job, one row per branch.

| Field | Data Type | Key / Constraint | Description |
|---|---|---|---|
| job_id | INT | PRIMARY KEY, FOREIGN KEY, NOT NULL | References job_listings(id). |
| branch | VARCHAR(50) | PRIMARY KEY, NOT NULL | Branch allowed to apply. No rows for a job means all branches are allowed. |

## 2.5 Applications Table

**Purpose:** Stores each student's application to a job; resolves the Students–Job Listings many-to-many relationship.

| Field | Data Type | Key / Constraint | Description |
|---|---|---|---|
| id | INT | PRIMARY KEY, AUTO_INCREMENT, NOT NULL | Unique identifier of the application. |
| student_id | INT | FOREIGN KEY, NOT NULL | References students(id). |
| job_id | INT | FOREIGN KEY, NOT NULL | References job_listings(id). |
| status | VARCHAR(20) | NOT NULL, DEFAULT 'APPLIED' | Current status of the application. |
| applied_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP | Date and time of application. |
| updated_at | TIMESTAMP | NOT NULL, DEFAULT CURRENT_TIMESTAMP | Date and time of the last status change. |
| UNIQUE (student_id, job_id) | CONSTRAINT | UNIQUE | Prevents a student from applying to the same job twice. |

## 2.6 Interview Slots Table

**Purpose:** Stores interview rounds scheduled for an application.

| Field | Data Type | Key / Constraint | Description |
|---|---|---|---|
| id | INT | PRIMARY KEY, AUTO_INCREMENT, NOT NULL | Unique identifier of the interview round. |
| application_id | INT | FOREIGN KEY, NOT NULL | References applications(id). |
| round_no | INT | NOT NULL | Round number of the interview. |
| scheduled_at | TIMESTAMP | NOT NULL | Scheduled date and time. |
| venue | VARCHAR(150) | NOT NULL | Venue of the interview. |
| passed | BOOLEAN | NULL | Result of the round; NULL until evaluated. |
| notes | TEXT | NULL | Optional remarks about the round. |

## 2.7 Offers Table

**Purpose:** Stores offers made against an application.

| Field | Data Type | Key / Constraint | Description |
|---|---|---|---|
| id | INT | PRIMARY KEY, AUTO_INCREMENT, NOT NULL | Unique identifier of the offer. |
| application_id | INT | FOREIGN KEY, NOT NULL | References applications(id). |
| package_lpa | DECIMAL(6,2) | NOT NULL | Offered package in lakhs per annum. |
| offer_letter_key | VARCHAR(255) | UNIQUE, NOT NULL | Key referencing the offer-letter file in the object store. |
| accepted | BOOLEAN | NOT NULL, DEFAULT FALSE | Whether the student accepted the offer. |

## 3. Key Relationships and Constraints

| Relationship | Cardinality / Description |
|---|---|
| Companies → Job Listings | One company can post many job listings (1:N). |
| Job Listings → Job Allowed Branches | One job has many allowed branches (1:N). A job with no rows is open to all branches. |
| Students → Applications | One student can submit many applications (1:N). |
| Job Listings → Applications | One job can receive many applications (1:N). |
| Students ↔ Job Listings | Many-to-many relationship implemented through applications. |
| Applications → Interview Slots | One application can have many interview rounds (1:N). |
| Applications → Offers | One application can have offers (1:N). |
| Applications (student_id, job_id) | UNIQUE: a student cannot apply to the same job twice. |
| Students (usn, email), Companies (company_name), Offers (offer_letter_key) | UNIQUE: each value identifies exactly one record. |
| Students (cgpa, active_backlogs), Job Listings (max_backlogs) | CHECK: cgpa between 0 and 10; backlog counts cannot be negative. |

## 4. Entity Relationship (ER) Diagram

The ER diagram below shows the entities, attributes, primary/foreign keys, relationship types, and cardinalities of the Talent Hub database. A single bar marks the “one” side and a crow’s foot marks the “many” side.

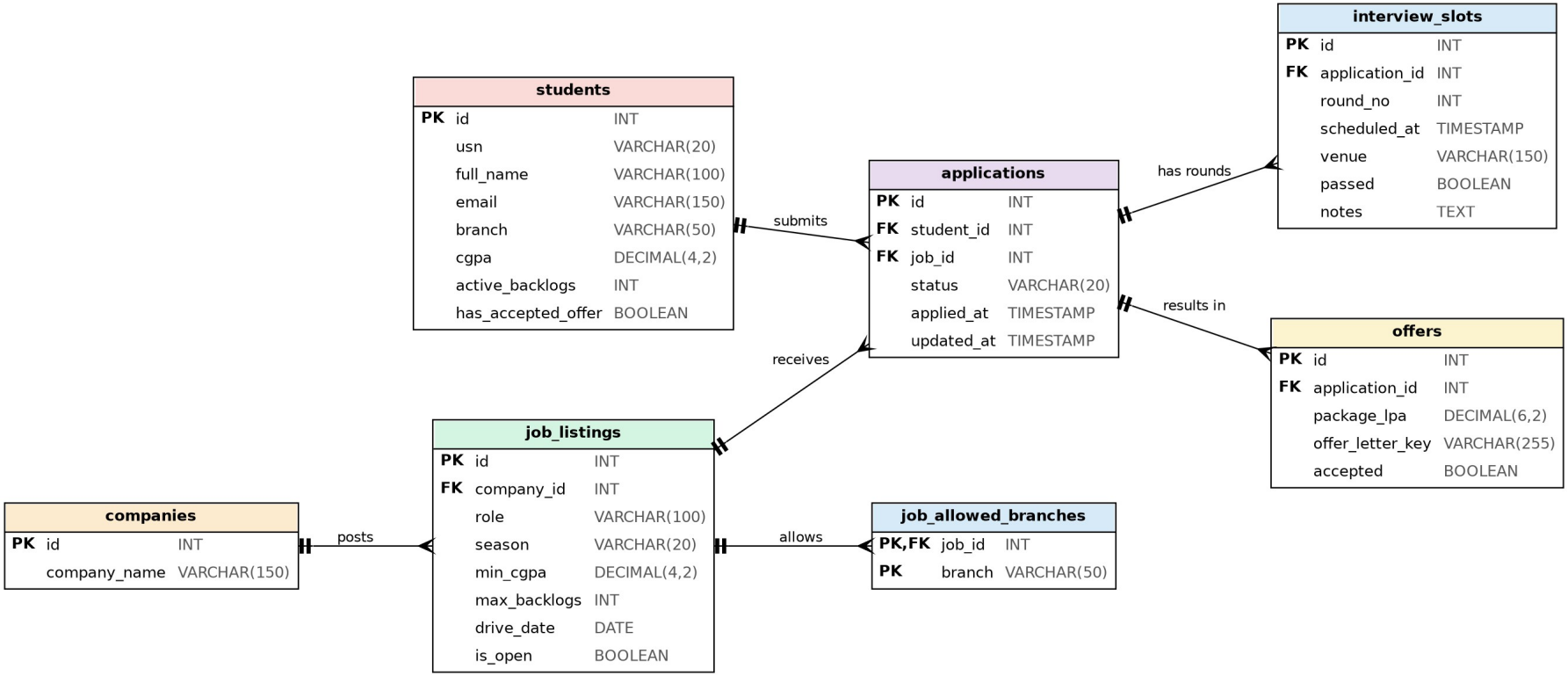



*Figure 1. Talent Hub Entity Relationship Diagram*

# 5. First Normal Form (1NF) – Talent Hub Database

## 5.1 1NF Requirement

The Talent Hub database is designed so that each table follows the basic conditions of First Normal Form:

- Every field contains a single value.
- A field does not contain a collection or list of values.
- The same attribute is not repeated in multiple columns.
- Each row can be uniquely identified using a primary key.
- Each column represents one specific type of information.

These conditions ensure that the data can be stored and queried consistently.

## 5.2 Checking the Talent Hub Tables

| Table | Atomic Values | Repeating Groups | Primary Key | 1NF Status |
|---|---|---|---|---|
| students | Yes | No | id | Satisfies 1NF |
| companies | Yes | No | id | Satisfies 1NF |
| job_listings | Yes | No | id | Satisfies 1NF |
| job_allowed_branches | Yes | No | (job_id, branch) | Satisfies 1NF |
| applications | Yes | No | id | Satisfies 1NF |
| interview_slots | Yes | No | id | Satisfies 1NF |
| offers | Yes | No | id | Satisfies 1NF |

## 5.3 Handling Multi-Valued Information

During the design of Talent Hub, some information could naturally contain multiple values. The original job_listings table held the eligible branches of a job as a list in one column. Instead of keeping those values in one field, a separate relation was created. The data below is illustrative.

### Example: Job Allowed Branches

A poor design would be:

| id | role | allowed_branches |
|---|---|---|
| 1 | Software Engineer | CSE, ISE, ECE |

Here, allowed_branches contains several values in one cell. The Talent Hub design separates this information:

### Job Listings

| id | company_id | role |
|---|---|---|
| 1 | 1 | Software Engineer |

### Job Allowed Branches

| job_id | branch | Note |
|---|---|---|
| 1 | CSE | |
| 1 | ISE | |
| 1 | ECE | |
| 2 | (no rows) | Job 2 is open to all branches |

Each cell now contains only one value, and the composite key (job_id, branch) identifies each row.

## 5.4 Multiple Job Listings per Company

A company can post several jobs. The jobs are not stored together in a companies record. Every job is a separate tuple in job_listings.

| id | company_id | role |
|---|---|---|
| 1 | 1 | Software Engineer |
| 2 | 1 | Data Analyst |
| 3 | 2 | QA Engineer |

Therefore, there is no field such as: jobs = Software Engineer, Data Analyst. The job_listings relation handles these values individually.

## 5.5 Multiple Applications per Student

A student can apply to many jobs. Instead of storing several job IDs in one student record, each application is a separate row in applications.

| id | student_id | job_id | status |
|---|---|---|---|
| 1 | 1 | 1 | APPLIED |
| 2 | 1 | 3 | APPLIED |
| 3 | 2 | 1 | APPLIED |

The pair (student_id, job_id) is UNIQUE, so the same application cannot be repeated.

## 5.6 Multiple Interview Rounds

An application can go through several interview rounds. Each round is stored as its own row in interview_slots.

| id | application_id | round_no | scheduled_at | passed |
|---|---|---|---|---|
| 1 | 1 | 1 | 2026-11-10 10:00 | TRUE |
| 2 | 1 | 2 | 2026-11-10 14:00 | NULL |

Round-specific data such as venue, result and notes is stored with the round it belongs to.

## 5.7 Multiple Offers

An application can have offers. Each offer is a separate row in offers, so package, offer-letter reference and acceptance are stored per offer.

| id | application_id | package_lpa | offer_letter_key | accepted |
|---|---|---|---|---|
| 1 | 1 | 6.50 | offers/1.pdf | FALSE |
| 2 | 3 | 7.00 | offers/2.pdf | FALSE |

## 5.8 Student and Job Relationship

A student can apply to many jobs and a job can receive many applications. Instead of storing lists on either side, the applications table represents each pairing separately and carries the attributes of the pairing: status, applied_at and updated_at.

## 5.9 Result After Applying 1NF

Talent Hub required one structural correction at this stage: the list in job_listings.allowed_branches was moved to job_allowed_branches. All other multi-valued data was already stored in separate relations. The final 1NF relations are:

- students
- companies
- job_listings
- job_allowed_branches
- applications
- interview_slots
- offers

### Final Conclusion

**The Talent Hub database satisfies First Normal Form (1NF).** Every attribute contains a single value, repeating groups have been avoided, multi-valued relationships are represented through separate tables, and every relation has a primary key.

This is the 1NF stage only. Further decomposition based on partial dependencies and transitive dependencies belongs to 2NF and 3NF, respectively.

# SECOND NORMAL FORM (2NF) – TALENT HUB DATABASE

## 2NF Requirement

A relation is in Second Normal Form (2NF) when it is already in First Normal Form (1NF) and every non-key attribute is fully dependent on the entire primary key. A partial dependency must not exist, meaning a non-key attribute cannot depend on only part of a composite primary key.

In the Talent Hub schema, most relations have a single-column primary key. Partial dependency can occur only in relations with composite keys, so these relations require specific verification.

## 1. 2NF Analysis of Talent Hub Tables

| Table | Primary Key | Composite Key? | Partial Dependency | 2NF Status |
|---|---|---|---|---|
| Students | id | No | None possible | Satisfies 2NF |
| Companies | id | No | None possible | Satisfies 2NF |
| Job_Listings | id | No | None possible | Satisfies 2NF |
| Job_Allowed_Branches | (job_id, branch) | Yes | None | Satisfies 2NF |
| Applications | id | No | None possible | Satisfies 2NF |
| Interview_Slots | id | No | None possible | Satisfies 2NF |
| Offers | id | No | None possible | Satisfies 2NF |

## 2. Composite-Key Table: Job_Allowed_Branches

The Job_Allowed_Branches relation uses the composite primary key (job_id, branch). It has no non-key attributes, so nothing can depend on only part of the key.

```
(job_id, branch) → no non-key attributes
```

| job_id | branch | Non-key attributes |
|---|---|---|
| 1 | CSE | None |
| 1 | ISE | None |
| 2 | ECE | None |

## 3. Composite Candidate Key: Applications (student_id, job_id)

The Applications relation has the single-column primary key id, and (student_id, job_id) is a candidate key enforced by UNIQUE. The remaining attributes describe one student applying to one job:

```
(student_id, job_id) → id, status, applied_at, updated_at
```

The dependency is on the complete pair. Status and timestamps cannot be determined from student_id alone, because a student applies to many jobs, nor from job_id alone, because a job receives many applications.

| student_id | job_id | status | applied_at | updated_at |
|---|---|---|---|---|
| 1 | 1 | APPLIED | 2026-10-01 | 2026-10-01 |
| 1 | 3 | APPLIED | 2026-10-02 | 2026-10-02 |
| 2 | 1 | APPLIED | 2026-10-02 | 2026-10-03 |

**Therefore:** student_id → status is not sufficient; job_id → status is not sufficient; (student_id, job_id) → status is valid. The same principle applies to applied_at and updated_at.

## 4. Tables with Single-Attribute Primary Keys

The remaining Talent Hub relations use a single-column primary key. A partial dependency cannot occur when the primary key contains only one attribute because there is no smaller part of the key.

| Table | Primary Key | Functional Dependency |
|---|---|---|
| Students | id | id → usn, full_name, email, branch, cgpa, active_backlogs, has_accepted_offer |
| Companies | id | id → company_name |
| Job_Listings | id | id → company_id, role, season, min_cgpa, max_backlogs, drive_date, is_open |
| Applications | id | id → student_id, job_id, status, applied_at, updated_at |
| Interview_Slots | id | id → application_id, round_no, scheduled_at, venue, passed, notes |
| Offers | id | id → application_id, package_lpa, offer_letter_key, accepted |

## 5. 2NF Result

After checking the dependencies, no table in the Talent Hub schema requires decomposition for 2NF. The only relation with a composite primary key, Job_Allowed_Branches, has no non-key attributes, and Applications depends on the whole (student_id, job_id) pair. All other relations use single-attribute primary keys and therefore cannot contain partial dependencies.

Student facts live in students, job and company facts in job_listings and companies, and applications stores only facts about one student applying to one job. A flat table mixing these would repeat names, roles and company names and cause update, insert and delete anomalies.

## Final 2NF Schema

- students
- companies
- job_listings
- job_allowed_branches
- applications
- interview_slots

- offers

## Conclusion

The Talent Hub database satisfies Second Normal Form (2NF). The schema is already in 1NF, and no non-key attribute depends on only part of a composite primary key. Therefore, no additional decomposition is required at the 2NF stage. Analysis of transitive dependencies belongs to Third Normal Form (3NF).

# THIRD NORMAL FORM (3NF) – TALENT HUB DATABASE

## 1. 3NF Requirement

A relation is in Third Normal Form (3NF) when it is already in 2NF and no non-key attribute depends on another non-key attribute. Every non-key attribute must depend directly on the primary key.

**Primary Key → Non-Key Attribute**

**Not: Primary Key → Non-Key Attribute → Another Non-Key Attribute**

## 2. 3NF Analysis of Talent Hub Tables

| Table | Primary Key | Transitive Dependency | 3NF Status |
|---|---|---|---|
| Students | id | None | Satisfies 3NF |
| Companies | id | None | Satisfies 3NF |
| Job_Listings | id | None | Satisfies 3NF |
| Job_Allowed_Branches | (job_id, branch) | None | Satisfies 3NF |
| Applications | id | None | Satisfies 3NF |
| Interview_Slots | id | None | Satisfies 3NF |
| Offers | id | None | Satisfies 3NF |

Branch is treated as a plain attribute of a student with no dependency on other non-key columns, because there is no evidence of a branch → anything dependency.

## 3. Example: Company Information

If company_name were stored directly in Job_Listings, the dependencies would be:

```
id → company_id
company_id → company_name
Therefore: id → company_id → company_name
```

This would create a transitive dependency. Talent Hub avoids this by keeping company details in a separate Companies table (illustrative data).

### Job Listings

| id | company_id | role |
|---|---|---|
| 1 | 1 | Software Engineer |
| 2 | 1 | Data Analyst |
| 3 | 2 | QA Engineer |

### Companies

| id | company_name |
|---|---|
| 1 | TechCorp |
| 2 | DataSoft |

## 4. Example: Student Information

Application records store student_id as a foreign key, while the student's name, branch and CGPA are maintained only in Students. This avoids the dependency: application id → student_id → cgpa.

### Applications

| id | student_id | job_id | status |
|---|---|---|---|
| 1 | 1 | 1 | APPLIED |
| 3 | 2 | 1 | APPLIED |

### Students

| id | full_name | branch | cgpa | active_backlogs |
|---|---|---|---|---|
| 1 | Rahul Sharma | CSE | 8.20 | 1 |
| 2 | Priya Patil | ISE | 7.80 | 0 |

## 5. Other 3NF Design Decisions

The same separation principle is followed throughout the database. Tables store foreign keys to related entities rather than duplicating descriptive information from those entities.

- Job_Listings store company_id instead of repeating the company name.
- Job_Allowed_Branches store job_id instead of repeating job details.
- Applications store student_id and job_id instead of repeating student and job details.
- Interview_Slots store application_id instead of repeating application, student or job details.
- Offers store application_id instead of repeating application details; the offer-letter file itself is kept in the object store and referenced by offer_letter_key.

## 6. Functional Dependency Structure

**STUDENTS :** id → usn, full_name, email, branch, cgpa, active_backlogs, has_accepted_offer

**COMPANIES :** id → company_name

**JOB_LISTINGS :** id → company_id, role, season, min_cgpa, max_backlogs, drive_date, is_open

**JOB_ALLOWED_BRANCHES :** (job_id, branch) → no non-key attributes

**APPLICATIONS :** id → student_id, job_id, status, applied_at, updated_at

**INTERVIEW_SLOTS :** id → application_id, round_no, scheduled_at, venue, passed, notes

**OFFERS :** id → application_id, package_lpa, offer_letter_key, accepted

## 7. Final 3NF Schema

- students
- companies
- job_listings
- job_allowed_branches
- applications
- interview_slots
- offers

## 8. Final Conclusion

The Talent Hub database satisfies Third Normal Form (3NF). The schema is already in 2NF, and no transitive dependencies remain between non-key attributes. Descriptive information such as company details is stored in a separate relation and referenced through foreign keys.

The normalized design reduces data redundancy, helps prevent update anomalies, preserves referential integrity, and provides a clean structure for the Talent Hub application.

One redundancy remains by design: students.has_accepted_offer and offers.accepted can hold overlapping information. This is not a normal-form violation, but the application or service layer must keep the two consistent, or has_accepted_offer must be derived from offers (see Lab 4, Part G).

# 9. BOYCE-CODD NORMAL FORM (BCNF) – TALENT HUB DATABASE

## 1. BCNF Requirement

A relation is in Boyce-Codd Normal Form (BCNF) when, for every non-trivial functional dependency X → Y, the determinant X must be a superkey of the relation. BCNF is stricter than 3NF. The Talent Hub database is therefore checked after establishing 3NF.

## 2. BCNF Analysis of Talent Hub Tables

| Table | Determinant(s) | Superkey? | BCNF Status |
|---|---|---|---|
| Students | id | Yes | Satisfies BCNF |
| Students | usn | Yes, UNIQUE + NOT NULL | Satisfies BCNF |
| Students | email | Yes, UNIQUE + NOT NULL | Satisfies BCNF |
| Companies | id | Yes | Satisfies BCNF |
| Companies | company_name | Yes, UNIQUE + NOT NULL | Satisfies BCNF |
| Job_Listings | id | Yes | Satisfies BCNF |
| Job_Allowed_Branches | (job_id, branch) | Yes | Satisfies BCNF |
| Applications | id | Yes | Satisfies BCNF |
| Applications | (student_id, job_id) | Yes, UNIQUE | Satisfies BCNF |
| Interview_Slots | id | Yes | Satisfies BCNF |
| Offers | id | Yes | Satisfies BCNF |
| Offers | offer_letter_key | Yes, UNIQUE + NOT NULL | Satisfies BCNF |

## 3. Students Table

The main dependency is id → usn, full_name, email, branch, cgpa, active_backlogs, has_accepted_offer. Since id is the primary key, it is a superkey. Because usn and email are UNIQUE and NOT NULL, each also uniquely identifies a student and acts as a candidate key. No BCNF violation exists.

## 4. Companies Table

The dependencies are id → company_name and company_name → id. id is the primary key and company_name is UNIQUE and NOT NULL, so both determinants are candidate keys. The relation satisfies BCNF.

## 5. Job Listings Table

id determines company_id, role, season, min_cgpa, max_backlogs, drive_date and is_open. Since id is the primary key, the determinant is a superkey and the relation satisfies BCNF.

## 6. Job Allowed Branches Table

The primary key is (job_id, branch) and the relation has no non-key attributes. The only determinant is the complete key, so the relation satisfies BCNF.

## 7. Applications Table

id determines student_id, job_id, status, applied_at and updated_at. The pair (student_id, job_id) is also UNIQUE, so it determines id and the remaining attributes and is a candidate key. Both determinants are superkeys, so the relation satisfies BCNF.

## 8. Interview Slots and Offers

Interview_Slots: id determines application_id, round_no, scheduled_at, venue, passed and notes; id is the primary key, so the relation satisfies BCNF.

Offers: id determines application_id, package_lpa, offer_letter_key and accepted. offer_letter_key is UNIQUE and NOT NULL, so it is also a candidate key. No non-superkey determinant exists, so the relation satisfies BCNF.

## 9. Final BCNF Schema

- students
- companies
- job_listings
- job_allowed_branches
- applications
- interview_slots
- offers

## 10. Final Conclusion

The Talent Hub database satisfies Boyce-Codd Normal Form (BCNF). Every non-trivial functional dependency has a determinant that is a superkey of its relation. No additional decomposition is required at the BCNF stage.

**Normalization result:** 1NF – Satisfied; 2NF – Satisfied; 3NF – Satisfied; BCNF – Satisfied.

## 10. UML Class Diagram

The UML class diagram below represents the Talent Hub tables as classes, showing their attributes and the multiplicities of the relationships between them. The only operation shown is checkEligibility, the function specified in Laboratory 4.

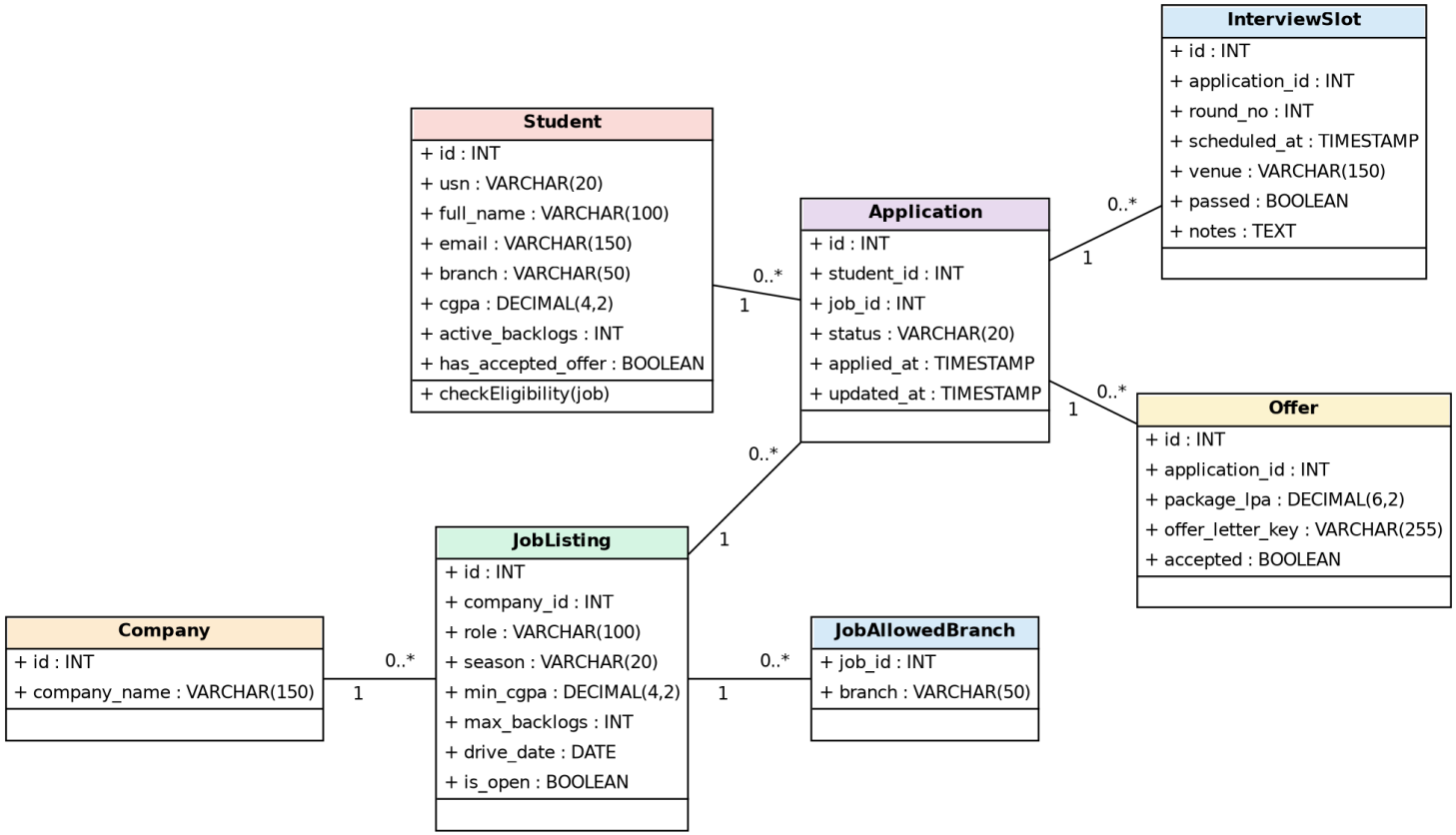



*Figure 2. Talent Hub UML Class Diagram*

# Laboratory 4
# AI-Assisted Detailed Design

*Data Design • Algorithm Design • Security Design | Consolidated Detailed Design Record (DDR)*

## Aim

To develop and validate implementation-ready detailed designs from the approved requirements and the Lab 3 architecture. AI proposes alternatives; the student identifies silent decisions and ACCEPTs, MODIFYs or REJECTs them to approve the final design.

## Learning Objectives

- Write a focused data-design prompt from approved requirements.
- Review AI-generated entities, attributes, keys, relationships and cardinalities.
- Review normalization up to 3NF using known functional dependencies.
- Specify one critical algorithm and review AI alternatives and trade-offs.
- Compare security controls for three important security needs.
- Identify silent decisions made by AI and consolidate them into one Detailed Design Record (DDR).

## Experiment Procedure

Method: Design Need → AI Alternatives → AI Silent Decision → Student Review (ACCEPT / MODIFY / REJECT) → Final Design.

| Part | Activity | Deliverable |
|---|---|---|
| A | Identify data needs, choose storage, and prompt AI for ER design. | Prompt + ER model |
| B | Review ER model and its silent decisions. | Corrected ER model + Silent Decisions |
| C | Prompt AI to normalize the relational schema up to 3NF. | Normalized schema |
| D | Review normalization silent decisions and finalize. | Final schema + Silent Decisions |
| E | Select one critical function; review AI alternatives; write pseudocode. | Algorithm design |
| F | Review three security decisions with two alternatives each. | Security design |
| G | Consolidate decisions and validate. | DDR + validation |

## Part A – Data Needs and Prompt for ER Model

### A1. Data needs and storage alternatives

Students, companies, job listings, allowed branches, applications, interview slots and offers are structured transactional data that need relationships, integrity and transactions. Offer letters are documents; the schema stores only a reference (offers.offer_letter_key, unique).

| Option | Design | When suitable |
|---|---|---|
| A | RDBMS only | Structured relationships, integrity and transactions. |
| B | RDBMS + file/object store | Relational records plus substantial documents or files. |
| C | RDBMS + specialized store | Approved search, vector, graph or other specialized workload. |

**AI silent decision:** Option B. **Student decision: ACCEPT B**, with the object store used only for offer letters, which the schema already references by key. Option C is rejected because no semantic-search requirement is approved. If the offer-letter file reference is not an approved requirement, fall back to Option A.

**Data map (two stores selected)**

| Data | Store | Source of truth |
|---|---|---|
| Students, companies, jobs, allowed branches, applications, interviews, offers (records) | RDBMS | RDBMS |
| Offer letter file | File/object store | File/object store; RDBMS keeps only offer_letter_key |

## A2. Prompt for ER model

**Prompt:** "Using the approved requirements below, propose entities, attributes, keys, relationships and cardinalities. Resolve M:N relationships and state PK/FK/UNIQUE constraints. Do not invent business rules; list assumptions."

**Requirements used with the prompt (summarised from the Talent Hub database design):**

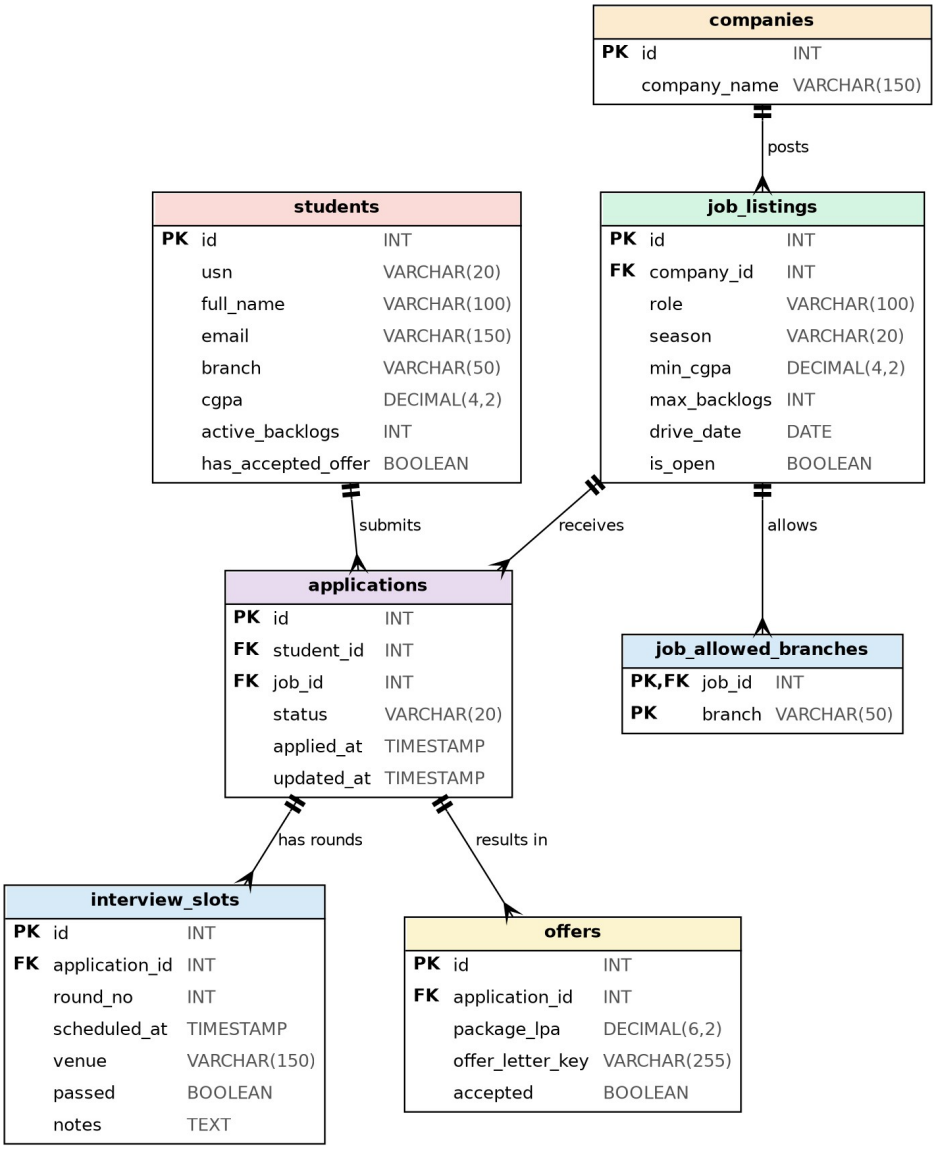
- Students are identified by a unique USN and email, and have a name, branch, CGPA (0 to 10), active backlogs and an offer-acceptance status.
- Companies are recruiting organisations with a unique name. A company posts many job listings.
- A job listing has a role, season, minimum CGPA, maximum backlogs, drive date and an open/closed flag. It can list allowed branches; a job with no allowed branches is open to all.
- A student can apply to many jobs but only once to each job. An application has a status and timestamps.
- An application can have many interview rounds, each with a round number, schedule, venue, result and notes.
- An application can have offers, each with a package, an offer-letter reference (unique) and an acceptance flag.

## Part B – ER Model Review and Silent Decisions

| ER Design Item | AI Decision / Assumption | Student Decision | Reason / Correction |
|---|---|---|---|
| Entity / attribute | Seven entities: students, companies, job_listings, job_allowed_branches, applications, interview_slots, offers, each with a surrogate integer id as primary key. | ACCEPT | Matches the TalentHub schema. Surrogate keys keep foreign keys simple. |
| | Keep students.has_accepted_offer as a stored column in addition to offers.accepted. | MODIFY | Overlapping data. Derive it from offers, or keep the two in sync in the service layer. |
| | Store allowed branches as a list inside job_listings. | REJECT | Multi-valued cell breaks 1NF. Moved to job_allowed_branches, one row per branch. |
| Relationship / cardinality | COMPANY 1:N JOB_LISTING; JOB_LISTING 1:N JOB_ALLOWED_BRANCH; STUDENT 1:N APPLICATION; JOB_LISTING 1:N APPLICATION; APPLICATION 1:N INTERVIEW_SLOT; APPLICATION 1:N OFFER. | ACCEPT | Follows the scenario flow: apply, interview, offer. |
| | STUDENT M:N JOB_LISTING resolved through APPLICATION. | ACCEPT | Each application is its own record with status and timestamps. |
| Key / constraint | UNIQUE (student_id, job_id) on applications. | ACCEPT | One application per student per job. |
| | UNIQUE usn, email, company_name, offer_letter_key; CHECK cgpa BETWEEN 0 AND 10; CHECK active_backlogs >= 0 and max_backlogs >= 0. | ACCEPT | Defined in the TalentHub schema. Assumption: confirm against the approved requirements. |

**Corrected ER model (text form)**

```
COMPANY (1) ──< (N) JOB_LISTING (1) ──< (N) JOB_ALLOWED_BRANCH
JOB_LISTING (1) ──< (N) APPLICATION (N) >── (1) STUDENT
APPLICATION (1) ──< (N) INTERVIEW_SLOT
APPLICATION (1) ──< (N) OFFER
```




*Corrected ER model of Talent Hub (7 entities, with allowed branches split out).*

## Part C – Prompt for Normalization

**Prompt:** "Given these relations and known functional dependencies, check 1NF, 2NF and 3NF. Identify anomalies, justify decompositions and give final relations with PKs/FKs. Do not invent dependencies; list assumptions."

**Relations and known functional dependencies given to the AI:**

| Relation | Attributes (PK marked) | Known Functional Dependency |
|---|---|---|
| STUDENTS | id (PK), usn, full_name, email, branch, cgpa, active_backlogs, has_accepted_offer | id → usn, full_name, email, branch, cgpa, active_backlogs, has_accepted_offer |
| COMPANIES | id (PK), company_name | id → company_name |

| Relation | Attributes (PK marked) | Known Functional Dependency |
|---|---|---|
| JOB_LISTINGS | id (PK), company_id, role, season, min_cgpa, max_backlogs, drive_date, is_open | id → company_id, role, season, min_cgpa, max_backlogs, drive_date, is_open |
| JOB_ALLOWED_BRANCHES | job_id (PK), branch (PK) | (job_id, branch) → no non-key attributes |
| APPLICATIONS | id (PK), student_id, job_id, status, applied_at, updated_at | id → student_id, job_id, status, applied_at, updated_at; (student_id, job_id) → id |
| INTERVIEW_SLOTS | id (PK), application_id, round_no, scheduled_at, venue, passed, notes | id → application_id, round_no, scheduled_at, venue, passed, notes |
| OFFERS | id (PK), application_id, package_lpa, offer_letter_key, accepted | id → application_id, package_lpa, offer_letter_key, accepted |

## Part D – Normalization Review and Silent Decisions

| Normalization Item | AI Decision / Assumption | Student Decision | Reason / Correction |
|---|---|---|---|
| Functional dependency / key | Each primary key determines all other attributes of its relation, including usn and email as candidate keys of students and (student_id, job_id) → id in applications. | ACCEPT | Each follows from the schema and the one-application-per-job rule. No extra dependencies assumed. |
| | Assumption: branch is a plain attribute of student with no dependency on other non-key columns. | ACCEPT | No evidence of a branch → anything dependency. Needed for the 3NF check. |
| Decomposition | 1NF: job_listings.allowed_branches holds a list. Split into job_allowed_branches (job_id, branch) with a composite key. | ACCEPT | Removes the multi-valued cell. A job with no rows is open to all branches. |
| | 2NF: the only composite key is (job_id, branch), which has no non-key columns; applications depends on the whole (student_id, job_id) pair. No change. | ACCEPT | No partial dependency exists. |
| | 3NF and BCNF: no non-key column determines another, and every determinant is a candidate key. No change. | ACCEPT | Each table holds facts about one entity only. |
| Final PK / FK / constraint | Seven relations with the keys listed in the final schema below. | ACCEPT | Result is in 3NF and also in BCNF. |

## Part E – Algorithm Design: checkEligibility(student, job)

| | |
|---|---|
| **Inputs** | student (cgpa, active_backlogs, branch, has_accepted_offer); job (min_cgpa, max_backlogs, is_open, allowed branches). |

| | |
|---|---|
| **Preconditions** | Student and job exist.<br>cgpa is between 0 and 10; backlogs >= 0. |
| **Output** | eligible (true/false) and a list of failure reasons. |
| **Constraints** | CGPA boundary is inclusive (cgpa >= min_cgpa).<br>Backlog boundary is inclusive (active_backlogs <= max_backlogs).<br>No allowed-branch rows means all branches; the job must be open. |
| **Edge cases** | cgpa equals min_cgpa.<br>Backlogs equal max_backlogs.<br>No branch rows.<br>Closed job.<br>Already-applied job (handled by the UNIQUE rule). |

**Alternatives and trade-offs**

| Option | Approach | Trade-off |
|---|---|---|
| A | Sequential fail-fast checks | Simple; reports only the first failure. |
| B | Evaluate all applicable checks | Reports every reason; slightly more logic. |
| C | Generic rule engine | Flexible but over-complex for four rules. |

**AI recommendation:** B. **Student decision: ACCEPT B.** A student should see all reasons they are ineligible. Example: Rahul (CGPA 8.2, 1 backlog) against min 7.0 and max 2 backlogs passes both. The has_accepted_offer (placement policy) check is included only if the approved requirements state it.

**Pseudocode**

```
FUNCTION checkEligibility(student, job)
   reasons = empty list
   IF job.is_open == false THEN add "Job is closed" to reasons
   IF student.cgpa < job.min_cgpa THEN add "CGPA below minimum" to reasons
   IF student.active_backlogs > job.max_backlogs THEN add "Too many active
   backlogs" to reasons
   branches = allowed branches of job
   IF branches is not empty AND student.branch NOT IN branches THEN add
   "Branch not allowed" to reasons
   IF placement policy approved AND student.has_accepted_offer == true THEN
   add "Already placed" to reasons
   RETURN (reasons is empty, reasons)
END FUNCTION
```

## Part F – Security Design

**Prompt:** “Using the approved requirements and Lab 3 architecture, propose two feasible controls for each of three important security needs. Compare advantages, limitations and assumptions. Recommend a control without inventing requirements.”

| Security Need | AI Alternatives and Trade-offs | Student Decision / Rationale |
|---|---|---|
| Authentication / authorization | A. Session-based authentication with server-side role checks: simple, server controls revocation, needs session storage.<br>B. Token-based authentication: stateless, suits separate clients, but expiry and revocation add complexity.<br>Trade-off: simplicity and revocation versus statelessness. | **Selected:** the mechanism already chosen in the Lab 3 architecture, with server-side role checks for Student, Company and Placement coordinator in the service layer.<br>Reason: three roles with different permissions; checks must not rely on the client. Authorization is not redesigned here. |
| Sensitive-data protection | A. Role-based field filtering of one response: fewer endpoints, but a filtering mistake can leak fields.<br>B. Role-specific API responses: stronger separation, but more endpoints to maintain.<br>Trade-off: simplicity versus stronger separation. | **Selected:** role-based field filtering in the service layer.<br>Reason: students' usn, email, cgpa and backlogs are exposed only to authorized roles; companies see only the fields they need for shortlisting. |
| Integrity / audit / abuse control | A. Log only sensitive changes (application status, interview results, offers): low volume.<br>B. Log all operations: complete history, but high volume and noise.<br>Trade-off: audit completeness versus volume. | **Selected:** authorize every status change by role, and log sensitive changes (status, offers, acceptance) using updated_at and an audit record.<br>Reason: protects placement records without adding an unapproved full audit requirement. Dedicated audit history only if required. |

## Part G – Consolidated Detailed Design Record (DDR)

| Design Area | AI Silent Decision | Human Decision | Action | Reason / Trade-off |
|---|---|---|---|---|
| Data | Option B: RDBMS + file/object store; seven-table schema in 3NF/BCNF. | RDBMS for all records; object store only for offer letters referenced by offer_letter_key; no vector or search store. | MODIFY | Avoids unsupported infrastructure. has_accepted_offer to be derived from offers or kept consistent. |
| Algorithm | checkEligibility with approach B (evaluate all rules). | Evaluate all rules and return every failure reason; inclusive CGPA and backlog boundaries. | ACCEPT | Students need all reasons. Placement-policy rule included only if approved. |
| Security | Role checks, controlled field exposure, selective audit. | Server-side role authorization, role-based field filtering, authorized status changes with logging of sensitive changes. | ACCEPT | Protects placement records with minimal added complexity. |

**DDR validation**

| Check | PASS / FAIL | Evidence / Correction |
|---|---|---|
| Consistent with approved requirements | PASS | Entities and rules come from the TalentHub scenario and schema; unapproved rules are marked as assumptions. |
| Consistent with Lab 3 architecture and ADRs | PASS | Router, service, repository and database layers kept; eligibility and authorization sit in the service layer. Confirm the authentication mechanism against the ADRs. |
| Unsupported AI assumptions removed or clarified | PASS | Vector store rejected; company_name uniqueness, offer-letter reference and placement-policy rule flagged for confirmation. |
| Sufficiently detailed for code generation | PASS | Keys, constraints, pseudocode with edge cases and security controls are specified. |

**Items to confirm against the approved requirements**

- Placement-policy rule: whether an already-placed student (has_accepted_offer = true) is ineligible.
- Offer-letter file reference: whether storing offer letters in an object store is an approved requirement (otherwise fall back to Option A).
- company_name uniqueness as an approved constraint.
- Authentication mechanism against the Lab 3 ADRs.
- Synchronisation or derivation of has_accepted_offer from offers.

## Final Submission

- Prompt used for ER modelling.
- Corrected ER model.
- Prompt/input used for normalization, including known functional dependencies.
- Final normalized relational schema up to 3NF.
- ER Silent-Decision table and Normalization Silent-Decision table.
- Storage alternative comparison and data map.
- Algorithm alternatives and pseudocode for one critical function.
- Three security decisions with alternatives, trade-offs and justified selections.
- One consolidated DDR with three human decisions and four validation checks.

**Final normalized relational schema (3NF, also satisfying BCNF):**

| Relation | Primary Key | Foreign Keys | UNIQUE / CHECK |
|---|---|---|---|
| students | id | — | UNIQUE: usn, email; CHECK: cgpa 0 to 10, active_backlogs >= 0 |

| Relation | Primary Key | Foreign Keys | UNIQUE / CHECK |
|---|---|---|---|
| companies | id | — | UNIQUE: company_name |
| job_listings | id | company_id → companies | CHECK: max_backlogs >= 0 |
| job_allowed_branches | (job_id, branch) | job_id → job_listings | No rows = all branches allowed |
| applications | id | student_id → students; job_id → job_listings | UNIQUE (student_id, job_id) |
| interview_slots | id | application_id → applications | — |
| offers | id | application_id → applications | UNIQUE: offer_letter_key |

**Boundary:** this laboratory does not redesign the Lab 3 architecture, invent requirements or generate code.